

Politechnika Warszawska

W Y D Z I A Ł   E L E K T R Y C Z N Y



# Dokumentacja Implementacyjna

Daniel Khadarouski

22 Marca 2026 r.

# Spis treści

|          |                                                   |          |
|----------|---------------------------------------------------|----------|
| <b>1</b> | <b>Architektura Projektu</b>                      | <b>3</b> |
| 1.1      | Ogólny model . . . . .                            | 3        |
| 1.2      | Logiczny podział na moduły . . . . .              | 3        |
| 1.3      | Przepływ danych . . . . .                         | 3        |
| 1.4      | Interakcja między modułami . . . . .              | 3        |
| <b>2</b> | <b>Technologie</b>                                | <b>3</b> |
| 2.1      | Język programowania . . . . .                     | 3        |
| 2.2      | Framework . . . . .                               | 4        |
| 2.3      | Narzędzia . . . . .                               | 4        |
| <b>3</b> | <b>Struktura Projektu</b>                         | <b>4</b> |
| 3.1      | Visual tree . . . . .                             | 4        |
| 3.2      | Opis modułów . . . . .                            | 4        |
| 3.3      | Logika porządku . . . . .                         | 5        |
| <b>4</b> | <b>Struktura Danych</b>                           | <b>5</b> |
| <b>5</b> | <b>Interfejsy</b>                                 | <b>5</b> |
| 5.1      | Interfejs modułu Parser(plik parser.h) . . . . .  | 5        |
| 5.2      | Interfejs modelu Graph(plik graph.h) . . . . .    | 6        |
| 5.3      | Interfejs modułu Layout (plik layout.h) . . . . . | 6        |
| 5.4      | Interfejs modułu Write (plik write.h) . . . . .   | 6        |
| <b>6</b> | <b>Zasady implementacji</b>                       | <b>6</b> |
| 6.1      | Zarządzanie Pamięcią . . . . .                    | 6        |
| 6.2      | Modułowość i enkapsulacja . . . . .               | 7        |
| 6.3      | Defense . . . . .                                 | 7        |
| 6.4      | Precyzja obliczeń . . . . .                       | 7        |
| 6.5      | Czytelność kodu . . . . .                         | 7        |

# 1 Architektura Projektu

## 1.1 Ogólny model

Program jest rozdzielony na niezależne moduły, gdzie każdy odpowiada za swoją część (input, rachunki, output). To pozwala na lekką wymianę np. jednego algorytmu na drugi bez zmian całej programy w związku z tym.

## 1.2 Logiczny podział na moduły

- **Moduł zarządzania danymi:** Pliki `graph.c` i plik nagłówkowy `graph.h` to są jądrem całej programy. One opowiadają za przechowywanie węzłów i krawędzi w pamięci.
- **Moduł wejścia:** Pliki `parser.c` i plik nagłówkowy `parser.h` to pliki odpowiadające za wczytywanie zewnętrznych plików i transformacja ich w wewnętrzny format danych.
- **Moduł obliczeń:** Pliki `layout.c` i plik nagłówkowy `layout.h` to pliki odpowiadające za obliczenia. One biorą `graph` i wylicza odpowiednie koordynaty.
- **Moduł wyjścia:** Pliki `write.c` i plik nagłówkowy `write.h` to pliki odpowiadające za transformację danych w format `.txt` lub `.bin`, który jest będzie zrozumiały użytkownikowi.

## 1.3 Przepływ danych

- **Początek:** Użytkownik nadaje ścieżkę do pliku przez `main`.
- **Parsing:** Moduł `parser` liczy krawędzi i wypełnia strukturę `graph`.
- **Processing:** Moduł `layout` wybiera odpowiedni nam algorytm (Spectral Layout lub Fruchterman-Reingold).
- **Koniec:** Moduł `write` zachowuje końcowe koordynaty (x,y) w plik wyjściowy.

## 1.4 Interakcja między modułami

Moduły komunikują się w pliku `main.c` za pomocą struktury `graph`, która przekazuje się za pomocą wskaźnika.

# 2 Technologie

## 2.1 Język programowania

C (standard C11). Język C był wybrany dla naszego projektu z powodu wymagań dotyczących wydajności obliczeniowych wybranych m.in. algorytmów Spectral Layout oraz Fruchterman-Reingold. Język C zapewnia wysoką przenośność kodu oraz precyzyjną kontrolę nad zarządzaniem pamięcią operacyjną przy przetwarzaniu dużych struktur gałowych.

## 2.2 Framework

Projekt nie wykorzystuje ciężkich frameworków,co ma na celu minimalizację narzutu systemowego.System opiera się na modułowej architekturze,wykorzystującej standardowe biblioteki języka C.

## 2.3 Narzędzia

- **GCC**:To główny kompilator z flagami optymalizacyjnymi oray ostrzeżeniami(np - wall),co pozwala na wczesne wykrywanie błędów w kodzie.
- **Valgrind**:Będzie wykorzystany na etapie testów w celu weryfikacji poprawności zarządzania pamięcią dynamiczną i eliminacji potencjalnych wycieków pamięci.
- **Git**:Git jest używany do zarządzanie pracą nad projektem.

## 3 Struktura Projektu

### 3.1 Visual tree

|          |   |                |             |
|----------|---|----------------|-------------|
| graph.c  | 2 | graph.c        | last week   |
| graph.h  |   | Create graph.h | last week   |
| layout.c | 4 | layout.c       | last week   |
| layout.h |   | layout.h       | last week   |
| main.c   | 1 | main.c         | 2 weeks ago |
| parser.c | 3 | parser.c       | last week   |
| parser.h |   | parser.h       | last week   |
| write.c  | 5 | write.c        | last week   |
| write.h  |   | write.h        | last week   |

### 3.2 Opis modułów

- **Main(plik main.c)**:Wyzywa funkcje z innych modułów w odpowiednim porządku.
- **Moduł Graph(pliki graph.c/.h)**:To jest fundamentem programu.Tutaj są opisane struktury node i edge.Funkcje tutaj zajmują się wydzieleniem pamięci,dodawaniem krawędzi,oczyszczeniem pamięci.
- **Moduł Parser(pliki parser.c/.h)**:Wczytuje textowy plik z liczbami i przekształcić go w zrozumiały programu obiekt graphu.
- **Moduł Layout(pliki layout.c/.h)**:Tutaj idą główne procesy obliczeniowe związane z wybraną nami funkcją.
- **Moduł Write(pliki write.c/.h)**:Bierze gotowe koordynaty i wpisuje te dane w plik wyjściowy,żeby było możliwe ich przeglądanie przez użytkownika.

### 3.3 Logika porządku

Projekt jest rozdzielony na moduły żeby kod był bardziej uporządkowany. Np jeśli my będziemy chcieli zmienić format pliku wejściowego, trzeba będzie naprawić tylko plik `parser.c`, a logikę algorytmów nie trzeba zmieniać. To pozwala na lekką zmianę kodu w ciągu projektu.

## 4 Struktura Danych

W programie zastosowano strukturalne podejście do reprezentacji grafu, wykorzystując dynamiczną alokację pamięci. Kluczowe struktury zdefiniowane w pliku `graph.h`.

- **Struktura node:** Reprezentuje pojedynczy wierzchołek graphu.

```
4     typedef struct{
5         int id
6         double x,y;
7     } node;
```

- **Struktura edge:** Definiuje połączenie między dwoma wierzchołkami.

```
typedef struct{
int i,v;
double weight;
} edge;
```

- **Struktura graph:** Główny kontener przechowujący kompletne dane o grafie.

```
typedef struct{
node *nodes;
edge *edges;
int node_count;
int edge_count;
} graph;
```

## 5 Interfejsy

### 5.1 Interfejs modułu Parser(plik `parser.h`)

Przyjmuje nazwę pliku z listą krawędzi. Wczytuje dane, tworzy strukturę graphu i zwraca wskaźnik na niej. Jeżeli pliku nie ma, zwraca NULL.

```
int readgraph(const char *filename, graph *g);
```

## 5.2 Interfejs modelu Graph(plik graph.h)

```
void init_graph(graph *g, int n, int e);
```

Inicjuje strukturę graphu. Ona rezerwuje miejsce w pamięci (przez malloc) dla maszyn krawędzi i wierzchołków, żeby algorytmy mogły z nimi działać.

```
void free_graph(graph *g);
```

Wyczyszcza pamięć dynamiczną, przekazaną pod węzły i krawędzie.

## 5.3 Interfejs modułu Layout (plik layout.h)

```
void spectral_layout(graph *g);
```

Oblicza koordynaty węzłów, wykorzystując własne wektory macierzy Laplasa. Odświeża pola x i y w strukturze każdego węzła.

```
void fruchterman_reingold(graph *g, int i);
```

Inicjuje iteracyjny siłowy algorytm. Wylicza siły przyciągania i odpychania, odnawiając pozycję węzłów potrzebną ilość razy.

## 5.4 Interfejs modułu Write (plik write.h)

```
int savetxt(const char *filename, graph *g);  
int savebin(const char *filename, graph *g);
```

Funkcje biorą końcowe koordynaty z struktury graphu i zapisuje ich w tekstowy plik (.txt lub .bin). Zwraca kod sukcesu lub błędu.

# 6 Zasady implementacji

## 6.1 Zarządzanie Pamięcią

Ze względu na specyfikę języka C, kluczową zasadą jest ścisła kontrola pamięci dynamicznej. Każda alokacja za pomocą malloc posiada odpowiadającą jej operację free. Implementacja funkcji freegraph zapewnia całkowite czyszczenie struktur danych przed zakończeniem programu

## 6.2 Modułowość i enkapsulacja

Kod został podzielony na niezależne moduły (pliki .h oraz .c). Interfejsy modułów są zdefiniowane w plikach nagłówkowych, co ukrywa szczegóły implementacyjne i ułatwia rozwój poszczególnych komponentów (np. dodanie nowego algorytmu layoutu) bez ingerencji w resztę systemu.

## 6.3 Defense

Program sprawdza poprawność operacji wejścia/wyjścia (np. otwarcie pliku) oraz sukces alokacji pamięci. W przypadku wystąpienia błędu, aplikacja wyświetla komunikat i kończy działanie w sposób kontrolowany.

## 6.4 Precyzja obliczeń

Do przechowywania współrzędnych węzłów oraz wyników operacji macierzowych wykorzystano typ double, co zapewnia wystarczającą dokładność dla iteracyjnych algorytmów rozmieszczania grafów.

## 6.5 Czytelność kodu

Zastosowano jednolite nazewnictwo zmiennych i funkcji oraz systematyczne wcięcia, co ułatwia analizę kodu i debugowanie.